

TP : création de beans session sans et avec état

1 - Bean Stateless

- Ecrire une interface qui propose une méthode salut, prenant une chaîne de caractères en paramètre, et retournant une chaîne de caractères.
- Implémenter cette interface dans un bean stateless : la méthode retourne un message de salutation pour la chaîne passée en paramètre (le nom de la personne à saluer).
- Tester dans un client lourd (voir page suivante pour la compilation et l'exécution)

2 - Bean Stateful

- Ecrire une interface qui propose la précédente méthode salut, ainsi qu'une autre méthode re_salut sans paramètre.
- Implémenter cette interface dans un bean stateful : la méthode re_salut retourne un message de salutation pour le nom transmis précédemment.

TP : création d'un bean entité

Configuration du TP

A - Créez une base de données nommée biblio qui contiendra la table et les données suivantes :

```
create table livre ( isbn char(4) not null primary key,  
                    titre char(20),  
                    dispo smallint default 1 not null, check (dispo in (0,1)));  
  
insert into livre values('111', 'Le petit prince', 1 );  
insert into livre values('222', 'Le rouge et le noir',1);  
insert into livre values('333', 'Germinal',1);
```

B - Configuration de Glassfish en ligne de commande pour accéder la base de données (2 lignes)

```
asadmin create-jdbc-connection-pool --datasourceclassname org.postgresql.ds.PGSimpleDataSource  
--restype javax.sql.ConnectionPoolDataSource --property  
portNumber=5432:password=pwdUser:user=nomUser:serverName=localhost:databaseName=biblio biblioPool  
  
asadmin create-jdbc-resource --connectionpoolid biblioPool jdbc/biblio
```

1 - Ecrire un Entity Bean Livre qui gère un livre. Définir les variables d'instance, et un constructeur à 2 arguments : l'isbn et le titre. A la création, le livre est disponible.

2 - Ecrivez une interface GestionLivre qui propose une méthode nouveauLivre à 2 paramètres. Ecrire son implémentation en un bean Stateless.

3 - Ecrivez le fichier de configuration de la persistance, créer l'archive puis déployez.

4 - Ecrivez une application cliente qui crée et supprime des beans.

5 - Transformer le bean de façon à ce qu'on puisse emprunter ou rendre des livres.

Le client : il est implémenté dans un répertoire qui n'a rien à voir avec celui de l'EJB.

Il faut copier l'archive jar dans le répertoire du client.

Si le nom de la classe client est *Client*, et le nom de l'archive serveur *TP.jar* :

Compilation du client : `javac -classpath TP.jar Client.java`

Exécution du client :

- sous Windows :

```
java -cp "%CLASSPATH%;TP.jar" --add-opens java.base/java.lang=ALL-UNNAMED Client
```

- sous Unix :

```
java -cp $CLASSPATH:TP.jar --add-opens java.base/java.lang=ALL-UNNAMED Client
```

Si toutefois vous avez des erreurs en exécutant le client, il faudra configurer la variable d'environnement `JAVA_TOOL_OPTIONS` pour qu'elle prenne cet ensemble de valeurs

```
--add-opens=java.base/java.util=ALL-UNNAMED
```

```
--add-opens=java.base/java.io=ALL-UNNAMED
```

```
--add-opens=java.base/java.lang=ALL-UNNAMED
```

```
--add-opens=java.management/javax.management.openmbean=ALL-UNNAMED
```

```
--add-opens=java.management/javax.management=ALL-UNNAMED"
```

- sous Windows :

```
setx JAVA_TOOL_OPTIONS "..."
```

- sous Unix :

```
export JAVA_TOOL_OPTIONS="...:..."
```

Configuration des propriétés de l'unité de persistance

1 - Le fichier `persistence.xml` a pour en-tête :

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<persistence xmlns=https://jakarta.ee/xml/ns/persistence
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" version="3.0"
```

```
xsi:schemalocation="https://jakarta.ee/xml/ns/persistence
```

```
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">
```

2 - Il faut y définir les propriétés suivantes :

`javax.persistence.jdbc.driver`

`org.postgresql.Driver`

`javax.persistence.jdbc.url`

`jdbc:postgresql://localhost:5432/biblio`

`javax.persistence.jdbc.user`

`nomUser`

`javax.persistence.jdbc.password`

`pwdUser`